
Compression d'image

Rapport final du projet de MAM3

ORSI Francesco, HAGHIGHI Camilia, JAM Lucile



POLYTECH
NICE SOPHIA



UNIVERSITÉ
CÔTE D'AZUR

SOMMAIRE

Introduction	3
I) Objectifs	3
II) Explications théoriques	4
III) Réalisations pratiques	6
III Carnet de bord	9
Conclusion	13
Références	14

Introduction

Dans le cadre de la première année du cycle ingénieur de Mathématiques Appliquées et Modélisation, nous avons été mis à l'ouvrage afin de réaliser un projet consistant à compresser une image png et jpeg avec la transformée de Fourier et la transformée de cosinus discrète. Ces outils mathématiques ont été au cœur de notre démarche pour aboutir à une compression efficace des images, qui consiste en deux étapes fondamentales : la compression de l'image et sa décompression.

I. Objectifs

Notre objectif premier consistait naturellement à maîtriser l'art de la compression d'images en utilisant des matrices de quantification, tout en relevant le défi de la décompression pour restaurer fidèlement l'image originale. Au-delà de cette ambition centrale, notre projet visait à consolider nos compétences en mathématiques et informatique acquises au cours de nos années et semestres précédents.

Cela représente une opportunité pour mettre en pratique nos connaissances théoriques, en particulier dans le domaine des transformations mathématiques appliquées aux images. De plus, ce projet nous a permis de nous familiariser avec des bibliothèques Python telles que Matplotlib et NumPy, renforçant ainsi notre maîtrise des outils essentiels en ingénierie.

Le travail en équipe constitue également un aspect essentiel de ce projet, nous offrant une occasion de perfectionner nos compétences en collaboration, avec un temps limité à 4 jours. La gestion du temps et la résolution de problèmes en temps réel sont importants, nous préparant ainsi pour des projets futurs et renforçant notre adaptabilité face aux défis. En résumé, notre projet de compression d'images représente bien plus qu'une simple application des concepts mathématiques. Il s'agit d'une opportunité d'apprentissage globale, englobant la maîtrise de compétences techniques, la collaboration en équipe et le développement de solutions innovantes pour des problèmes complexes.

II. Explications théoriques

Dans un premier temps, il a été primordial de trouver la matrice de passage en fréquentiel de la DCT2 pour pouvoir faire la compression. On a la formule :

$$D_{k,l} = \frac{1}{4} C_k C_l \sum_{i=0}^7 \sum_{j=0}^7 M_{i,j} \cos\left(\frac{(2i+1)k\pi}{16}\right) \cos\left(\frac{(2j+1)l\pi}{16}\right),$$

où $C_0 = \frac{1}{\sqrt{2}}$ et $C_k = 1$ si $k > 0$ (et idem en l).

On réécrit $D_{k,l}$ en fonction de i et j :

$$D_{k,l} = (PMT^T)_{k,l} = \sum_{i=0}^7 \sum_{j=0}^7 P_{k,i} M_{i,j} P_{l,j}$$

En comparant les 2 formules précédentes on individualise les termes dans les sommes et on trouve :

$$P_{k,i} M_{i,j} P_{l,j} = \frac{1}{4} C_k C_l M_{i,j} \cos\left(\frac{(2i+1)\pi k}{16}\right) \cos\left(\frac{(2j+1)\pi j}{16}\right)$$

En mettant les indices k et i ensemble et les l et j on arrive aux 2 formules suivantes :

$$P_{l,j} = \frac{1}{2} C_l \cos\left(\frac{(2j+1)\pi}{16}\right) \quad P_{k,i} = \frac{1}{2} C_k \cos\left(\frac{(2i+1)}{16} * k\pi\right)$$

On pose $k=l$ et $i=j$, ainsi on a trouvé comment calculer P , il ne reste qu'à implémenter une double boucle pour les sommes et de mettre la formule trouvée ci-dessus.

P est orthogonale donc on vérifie que $Id=P^tP$

Ensuite il faut se concentrer sur chaque bloc de 8×8 de l'image car notre matrice de quantification est une matrice carrée de 8 lignes et 8 colonnes. Notre matrice de quantification le cœur du processus de compression. En divisant terme à terme la matrice D (résultat de la DCT2) par la matrice Q et en arrondissant les résultats à la partie entière, on réduit le nombre de bits nécessaires pour représenter chaque coefficient. Cette étape est justifiée par le fait que certains coefficients, en particulier ceux correspondant aux hautes fréquences, peuvent être considérés comme moins perceptibles à l'œil humain et peuvent donc être quantifiés avec moins de précision.

En soustrayant 128 aux intensités de chaque pixel, ça va nous servir à centrer les valeurs autour de zéro. Cela permet d'éliminer le terme constant dans la transformée, simplifiant ainsi les calculs.

Le taux de compression est calculé en comparant le nombre de coefficients non nuls dans la matrice compressée par rapport au nombre total de coefficients dans la matrice originale. Une matrice compressée avec beaucoup de zéros est dite "creuse", ce qui indique une réduction de l'information.

Ensuite on va décompresser l'image, ça consiste à appliquer l'inverse du processus de compression fait auparavant. C'est-à-dire qu'on doit multiplier les coefficients quantifiés par la matrice Q et appliquer la transformée inverse pour retrouver une version décompressée de l'image.

Pour calculer l'erreur de compression, on fait la somme des valeurs absolues (norme de (matrice de décompression-matrice de la couleur))/(norme de (matrice de la couleur)). On parle ici de norme L2 relative.

De plus, on a fait plusieurs recherches pour trouver différentes matrices de quantification. D'une part on a remarqué que si on multipliait tous nos coefficients par des valeurs de plus en plus grandes, le taux de compression devenait de plus en plus élevé et ainsi la qualité de plus en plus mauvaise. Or, on est pas sûrs que cela soit une bonne idée de faire ça car cela pourrait mal traiter les hautes et basses fréquences.

En effet, il est important de savoir que la matrice de quantification Q est conçue de manière à attribuer des valeurs plus faibles aux fréquences de haute qualité visuelle et des valeurs plus élevées aux fréquences de moindre importance visuelle. Cela permet de concentrer l'information visuelle essentielle dans les basses fréquences et de réduire la contribution des hautes fréquences, ce qui est souvent perçu comme du bruit visuel.

On a donc trouvé deux matrices de quantification différentes de celle donnée :

$$Q_{highquality} = \begin{pmatrix} 2 & 1 & 1 & 2 & 2 & 4 & 5 & 4 \\ 1 & 1 & 2 & 3 & 2 & 6 & 6 & 6 \\ 2 & 2 & 2 & 2 & 4 & 9 & 8 & 7 \\ 2 & 2 & 2 & 2 & 5 & 8 & 9 & 7 \\ 2 & 2 & 4 & 5 & 7 & 12 & 11 & 9 \\ 3 & 4 & 7 & 9 & 11 & 11 & 10 & 8 \\ 5 & 7 & 8 & 9 & 10 & 12 & 12 & 10 \\ 7 & 8 & 9 & 10 & 11 & 10 & 10 & 10 \end{pmatrix}$$

$$Q_{lowquality} = \begin{pmatrix} 64 & 55 & 50 & 64 & 96 & 160 & 204 & 244 \\ 55 & 56 & 66 & 80 & 111 & 156 & 160 & 140 \\ 66 & 65 & 80 & 120 & 160 & 228 & 280 & 224 \\ 75 & 82 & 110 & 145 & 204 & 348 & 320 & 248 \\ 92 & 112 & 160 & 192 & 224 & 364 & 344 & 260 \\ 120 & 175 & 220 & 248 & 300 & 384 & 404 & 332 \\ 260 & 320 & 400 & 420 & 440 & 484 & 480 & 404 \\ 392 & 520 & 520 & 520 & 560 & 500 & 520 & 520 \end{pmatrix}$$

(Toutes les valeurs supérieures à 127 on peut les mettre à 127)

La matrice de quantification fournie pour notre projet est conçue de manière à avoir des valeurs plus élevées dans les coins inférieurs droits. Cette conception a pour effet que lors de la division par la matrice Q, les termes correspondant aux hautes fréquences, situés dans les coins inférieurs droits, sont largement annulés. En revanche, les basses fréquences, représentant généralement l'essentiel de l'information visuelle, sont préservées. Ainsi, la matrice de quantification de "high quality" générera une image moins compressée que celle avec la matrice Q d'origine, offrant ainsi une meilleure qualité visuelle en raison de ses coefficients relativement bas. À l'inverse, la matrice de quantification de "low quality" produira une image beaucoup plus compressée que celle obtenue avec la matrice Q d'origine, entraînant une réduction de la qualité visuelle.

III. Réalisations pratiques

Après avoir calculé les taux et normes relatives, un problème s'est formé lorsqu'on a voulu compiler le code. On avait des messages d'erreurs comme quoi certaines valeurs sortaient de l'intervalle [0,1]. Il a fallu donc modifier ces valeurs pour les approcher de 1 ou 0 en fonction de la valeur de celles-ci.

En faisant chaque couleur étape par étape, on a obtenu 3 photos suivantes selon R,G ou B :

R :



G :



B :



En compilant notre programme avec notre Q normal on obtient une image avec : un taux de compression à 0.892329 et une erreur de compression à : 0.099887. Notre programme s'effectue en 0.4446258 secondes. L'image obtenue est :



En compilant notre programme avec notre Q pour une meilleur qualité on obtient une image avec : un taux de compression plus bas, à 0.7282342 et une erreur de compression, plus basse également à : 0.018493. Notre programme s'effectue en 0.4791350 secondes. L'image obtenue est :



En compilant notre programme avec notre Q pour une moins bonne qualité on obtient une image avec : un taux de compression plus élevé, à 0.95955231 et une erreur de compression, plus élevée également à : 0.192464. Notre programme s'effectue en 0.3698179 secondes.

L'image obtenue est :



Elle fait 508ko, nettement moins que les autres, avec le Q normal on a une image à 900ko.

Donc, on en arrive à la conclusion que plus les valeurs de notre matrice de quantification sont élevées pour les basses fréquences, plus l'image sera compressée tout en préservant la qualité visuelle. Cela s'explique par le fait que les hautes fréquences, responsables des détails fins, sont atténuées, ce qui permet d'atteindre un taux de compression plus élevé. Cependant, des valeurs trop élevées dans la matrice Q peuvent également entraîner une perte de certaines informations importantes, ce qui se traduit par une erreur de compression plus élevée.

Ensuite, pour tronquer l'image (en combinant avec notre matrice de quantification Q), on a testé notre programmes lorsque l'on garde des fréquences inférieures à 6 , on obtient une image à 922ko , compressée à 89,34199% avec un taux d'erreur de 10,580%:



Ensuite, on a testé pour des fréquences inférieures à 4, on obtient une image à 836ko mais compressée à 91,041% avec un taux d'erreur à 14,7519% :



Ensuite on a testé en gardant les fréquences inférieures à 2 : on obtient une image de 738ko, compressée à 94,91972% avec un taux d'erreur à 23,56 %:



On en déduit que moins on garde de fréquence, c'est-à-dire, plus on baisse le maximum des fréquences que l'on souhaite garder, plus notre image sera compressée.

CARNET DE BORD :

LUNDI 18 DECEMBRE

Durant cette journée nous avons pu nous familiariser avec les librairies numpy et matplotlib de Python.

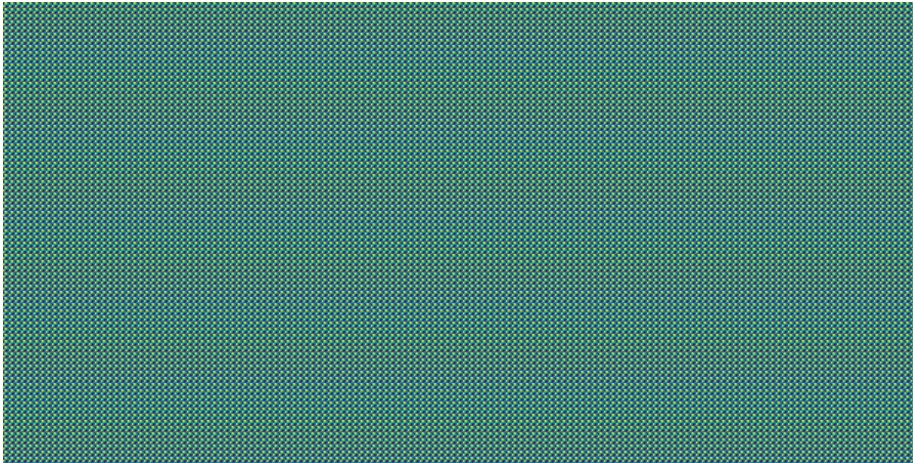
Nous avons réussi à trouver la formule pour trouver la matrice de changement de base.

Nous avons réussi à implémenter le code grâce à l'algorithme donné.

Nous avons debuggé tout le long ce qui nous a permis de vite régler les problèmes de codes que nous avons rencontrés (notamment de dimensions des tableaux), ou de problèmes de code mal écrit dans les boucles.

A la fin de la journée nous n'avons pu afficher que l'image compressée. Nous avons eu un petit problème lors de la compression que l'on résoudra le lendemain.

Image de l'image compressée (sans la décompression) que l'on obtient :



MARDI 19 DECEMBRE

Nous avons réglé le problème que nous rencontrions la veille. On n'arrivait à afficher seulement l'image compressée car on ne décompressait pas chaque bloc de 8 par 8 de l'image, il y avait un problème de code dans une boucle où une variable était réinitialisée plusieurs fois.

On a donc pu afficher la première « couleur »(rouge) et nous avons continué en faisant une boucle de 0 à 2 pour afficher les deux autres couleurs.

Le code jusque-là marche bien.

Nous avons donc cherché à optimiser notre code car il était un peu lent (7secondes d'exécution) avec la photo donnée de Polytech et aussi car l'image finale obtenue était plus volumineuse que l'image originale.

Nous avons trouvé que pour la taille de l'image c'était à cause des erreurs d'arrondis et nous avons largement baisser le temps d'exécution en enlevant certains calculs de la boucle de départ et aussi en mettant le calcul du taux en dehors de la double boucle de compression. Qui nous a fait gagner plus de 6 secondes sur le temps d'exécution pour finalement nous donner un temps d'exécutions d'environ 0.4 secondes.

On s'est aussi aperçus que notre programme supportait les photos de type .jpeg sauf qu'elle nous rendait un résultat en négatif et pour avoir le résultat final il fallait mettre à chaque valeur x de la matrice finale la valeur $1-x$. Ce qui nous rendait bien une image normale non inversée.

Lors de la prochaine séance nous allons chercher des matrices Q et un système de filtrage pour pouvoir faire varier notre taux de compression, et/ou aussi regarder pour intégrer les image jpeg.

MERCREDI 20 DECEMBRE

Nous avons cherché si d'autres matrices de quantification existent afin de varier la qualité de notre image. On en a trouvé quelques unes et on a remarqué que plus les coefficient de notre matrice sont élevées plus la qualité de la photo finale sera faible et inversement.

On a testé le filtrage en tronquant l'image, en gardant les fréquences inférieures à 2, 4 et 6.

Préparation de la présentation : oral, powerpoint, rapport.

Conclusion

En conclusion, notre projet de compression d'images avec la transformée de Fourier et la transformée de cosinus discrète a été enrichissant. En explorant les subtilités des techniques de compression, comme le filtrage des hautes fréquences ou encore l'utilisation d'une matrice de quantification, nous avons renforcé nos compétences en mathématiques appliquées et acquis une expérience pratique avec des bibliothèques Python essentielles. Ce projet nous a permis de relever des défis concrets, de collaborer efficacement en équipe, et de développer des solutions innovantes. En compressant et décompressant avec succès des images JPEG et PNG, nous avons démontré la pertinence de nos connaissances théoriques dans des applications du monde réel. Cette expérience intensive nous prépare de manière significative pour des projets futurs et nous laisse avec une compréhension approfondie des implications et des applications de la compression d'images.

Références

- <https://www.chireux.fr/mp/cours/Compression JPEG.pdf>
- <https://hal.science/hal-03858141/file/qtable.pdf>
-